# 1. Arrays in Python

## Definition

An **array** is a collection of elements stored in **continuous memory locations**, where:

- All elements are of the **same data type**
- Each element is accessed using an **index (position)**
- **Faster than list but less flexible**

Example:[10, 20, 30, 40]

## Syntax (array module)

```
import array
Variable = array.array(typecode, [initial_elements])
```

## Example Program

```
import array
#  Array Creation
arr = array.array('i', [10, 20, 30, 40])
print("Original Array:", arr)
#  Access Elements
print("First element:", arr[0])
print("Last element:", arr[-1])
#  Insert Elements
arr.append(50)
arr.insert(2, 25)
print("After Insertion:", arr)
# Update Elements
arr[1] = 99
print("After Update:", arr)
# Delete Elements
arr.remove(30)
arr.pop()
print("After Deletion:", arr)
#  Sum of Elements
total = sum(arr)
print("Sum of elements:", total)
```

# 2. Strings and Characters

In Python, a string is a sequence of characters written inside quotes. It can include letters, numbers, symbols and spaces.

- Python does not have a separate character type.
- A single character is treated as a string of length one.

- Strings are commonly used for text handling and manipulation.

Example: name = "Surya"

**Characteristics of String :**

- **Creation**: Strings can be defined using single quotes ('hello'), double quotes ("hello"), or triple quotes (""""hello""" or '''hello''') for multiline strings.
- **Immutability**: Once a string is created, it cannot be changed in place. Operations that appear to modify a string (like replace()) actually return a new string.
- **Indexing and Slicing**: Characters can be accessed using zero-based indexing (string[0]), and substrings can be extracted using slicing (string[start:end]).
- **Concatenation and Repetition**: Strings can be joined using the + operator and repeated using the * operator.

**<u>String Slices</u>**

Slicing is a way to extract a portion of a string by specifying the start and end indexes. The syntax for slicing is string[start:end], where start starting index and end is stopping index (excluded).

**Syntax**

string[start:end]

**Example:**

```
s = "python programming"
print(s[1:4])
print(s[:3])
print(s[3:])
print(s[::-1])
```

---

**Immutability**

Strings are immutable, which means that they cannot be changed after they are created. If we need to manipulate strings then we can use methods like concatenation, slicing or formatting to create new strings based on original.

**Example**:
**Strings cannot be changed**
text = "hello"
text[0] = 'H'  # Error

**But you can create a new string:**
text = "H" + text[1:]

---

# String Functions & Methods

Python provides various built-in methods to manipulate strings.

### Example:

text = "hello world”
print(len(text))  # 11

| Method | Example | Output |
|---|---|---|
| upper() | text.upper() | HELLO WORLD |
| lower() | text.lower() | hello world |
| title() | text.title() | Hello World |
| replace() | text.replace("hello","hi") | hi world |
| find() | text.find("world") | index |
| count | text.count("l") | number |

---

## String Module

The Python string module is part of the standard library and provides useful constants, a single utility function, and classes for common string-related operations. It is often used for tasks like input validation and custom text formatting.

**Key features:**
Validation: Check if characters in a string belong to specific sets.
Formatting: Format strings for better output or presentation.
Template Handling: Create templates with placeholders that can be dynamically filled**.**

**Example**:
```
import string
s = "capitalizes the first letter of each word"
print(string.capwords(s))
```

**Output**
Capitalizes The First Letter Of Each Word

---

# 3. Lists as Arrays

### Definition

- A list is a built-in data structure that can hold an ordered collection of items.
- They are Mutable (flexible) because you can modify them by adding, removing, or updating items.
- It can store mixed data types (integers, strings, booleans, even other lists). Elements are accessed by their position, starting at 0 for the first item and -1 for the last.

**Example**:
```
c = [1, 'hello', 3.14, True]
```

**Program**:
```
list1 = ["Rohan", "Physics", 21, 69.75]
print(list1)
print(list1[0])
```

**Output**:
['Rohan', 'Physics', 21, 69.75]
Rohan

### Syntax

list = [1, 2, 3, 4]

## 🔹 List Operations

lst = [1, 2, 3]

**append(item):** Adds a single item to the end of the list.
Eg: print(lst.append(4))

**extend(iterable):** Adds all items from an iterable (like another list or tuple) to the end of the current list.

**insert(index, item):** Inserts an item at a specified position.
print(lst.insert(1, 10))

**remove(item):** Removes the first occurrence of a specified item by value. Raises a ValueError if the item is not found.
print(lst.remove(2))

**del statement**: Deletes an item at a specific index or a slice of items
Eg: del lst

---

## 🔹 List Slices

List slicing in Python is a powerful way to extract a portion of a list (a sublist) by specifying a range of indices.

**Syntax**:
list_name[start:stop:step]

start (optional): The index where the slice begins. The default is 0.
stop (optional): The index where the slice ends
step (optional): The interval between elements in the slice. The default is 1

**Example**
lst = [10, 20, 30, 40]
print(lst[1:3])  # [20, 30]

---

## 🔹 List Methods

Python list methods are built-in functions that allow us to perform various operations on lists.

**copy():** Returns a shallow copy of the list.
**clear():** Removes all elements from the list.
**count():** Returns the number of times a specified element appears in the list.
**sort():** Sorts the list in ascending order (by default).
**reverse():** Reverses the order of the elements in the list.

**Example**:

```
a = [1, 2, 3]
# Create a copy of the list
b = a.copy()
print(b)      #output [1, 2, 3]

a = [3, 1, 2]
# Sort the list in ascending order
a.sort()
print(a)      #output [1, 2, 3]
```

---

## 🔹 List Loop

```
lst = [1, 2, 3]
for i in lst:
    print(i)
```

---

## 🔹 Mutability

Lists **can be changed**.This means that you can change the content or elements of a list after it has been created without changing its unique identity (memory address).

**Example**:

```
lst[200,150,350,400]
lst[0] = 100
print(lst)
```

---

## 🔹 Aliasing

In Python, aliasing occurs when multiple variables refer to the exact same mutable object in memory, such as a list.

**Example**:

```
a = [1,2]
b = a
b[0] = 99
print(a)  # also changes
```

---

## 🔹 Cloning Lists

Cloning a list in Python creates a new, independent list object.

**Example**

```
a = [1,2]
b = a.copy()
print(b)     # output [1,2]
```

---

## 🔹 List Parameters

List parameter is a variable defined in a function's signature that accepts a list as an argument when the function is called.

**Example**

```
def display(lst):
    print(lst)
display([1,2,3])
```

---

# List Comprehension

List comprehension in Python is a concise and efficient way to create a new list by applying an expression to each item in an existing iterable, optionally filtering the elements based on a condition. It provides a more readable alternative to a traditional for loop with list.append() calls

### Syntax

[new_list = [expression for item in iterable if condition]

**expression**: The value or operation to apply to each item
**item**: A variable representing the current element in the iteration.
**iterable**: sequence (e.g., list, tuple, range, string) to iterate over.
**condition** (optional): A filter that includes only items for which the condition evaluates to true.

### Example Squares of numbers

```
squares = [x**2 for x in range(10)]
print(squares)
```

**Example**: Filter even numbers

```
evens = [n for n in numbers if n % 2 == 0]
```

---

# 4. Tuples

### Definition

- Tuples are similar to lists, but unlike lists, they cannot be changed after their creation (i.e., they are immutable).
- The main characteristics of tuples are being ordered,  immutable. and allow duplicate values.
- A single tuple can contain items of different data types (e.g., strings, integers, and floats).

**Program**:

```
thistuple = ("apple", "banana", "cherry", "apple", "cherry")
print(thistuple)
```

**Output**:

```
('apple', 'banana', 'cherry', 'apple', 'cherry’)
```

---

### Syntax

```
t = (1, 2, 3)
```

---

## Accessing Values

In Python, you can access values in a tuple using indexing, slicing, unpacking, or by looping through its elements

**Example**: Indexing

```
tup = (10, 20, 30, 40, 50)
# Access the first item
print(tup[0])        #output : 10
```

---

## 🔹 Tuple Assignment

Tuple assignment in Python is a powerful feature that allows you to assign multiple values to multiple variables simultaneously in a single line of code.

**Example**:

```
a, b, c = 1, 2, 3
print(f"a: {a}, b: {b}, c: {c}")
# Output: a: 1, b: 2, c: 3
```

---

## 🔹 Tuple as Return Value

```
def fun():
    return 1, 2
x, y = fun()
```

---

## 🔹 Tuple Operations

Operations on tuples are generally those that do not modify the original tuple but rather return new ones

**Example**

```
t1 = (1,2)
t2 = (3,4)
print(t1 + t2)
```

---

## 🔹 Built-in Functions

Python's general built-in functions can operate on tuples, such as

- len(): Returns the total number of items
- max(): Returns the largest value
- min(): Returns the smallest value
- sum(): Adds all numeric elements
- sorted(): Returns a tuple in ascending order.

**Example**

```
t = (1,2,3)
print(len(t))
print(max(t))
```

---

# 5. Dictionaries

### Definition

- A dictionary (dict) in Python is a built-in, mutable (changeable) data type that stores data in the form of key–value pairs,
- Each item in a dictionary consists of a unique key mapped to a corresponding value.
- It allows fast retrieval, insertion, and modification of data based on keys.

**Example**:

my_dict = { “ name “: “ Alice ” , “ age ” : 30 }

**Program**:

```
person = {
    "name": "Alice",
    "age": 25,
    "city": "Paris"
}
print(person["name"])
```

**Output** :

Alice

---

### Syntax

d = {"name": "Sri", "age": 20}

---

## 🔹 Operations

```
print(d["name"])

d["age"] = 21
d["city"] = "Madurai"
```

---

## 🔹 Methods

```
print(d.keys())
print(d.values())
print(d.items())
```

---

# 6. Sum of Array

### Example

```
import array
arr = [10, 20, 30]
total = sum(arr)
print(total)
```

---

# 7. Linear Search

**Definition**
A linear search in Python iterates through each element of a list to find a target value, returning the index if found, or -1 otherwise.

### Program

```
arr = [10, 20, 30]
key = 20
for i in range(len(arr)):
    if arr[i] == key:
        print("Found at index", i)
```

---

# 8. Binary Search

### Definition

Search in **sorted array** using divide and conquer.

---

### Program

```
arr = [10, 20, 30, 40]
low = 0
high = len(arr)-1
key = 30

while low <= high:
    mid = (low + high) // 2
    if arr[mid] == key:
        print("Found")
        break
    elif arr[mid] < key:
        low = mid + 1
    else:
        high = mid - 1
```

---

# 9. Bubble Sort

### Definition

Sort by repeatedly swapping adjacent elements.

---

### Program

```
arr = [5, 3, 2, 4]

n = len(arr)
```

```
for i in range(n):
    for j in range(0, n-i-1):
        if arr[j] > arr[j+1]:
            arr[j], arr[j+1] = arr[j+1], arr[j]

print(arr)
```

---

# Unit 4

## 1.FILE

In Python, a file is a named location on disk used to permanently store data. The process of interacting with files (creating, reading, writing, etc.) is called file handling, which is a core concept for data persistence.

**File Operations**

The general sequence for file operations is:

1. Open the file using open().
2. Operate on the file (read or write data).
3. Close the file using close()

**File Modes**

| Mode | Description |
|---|---|
| 'r' | Read (default). Opens the file for reading. Raises an error if the file does not exist. |
| 'w' | Write. Opens the file for writing. Creates a new file if it doesn't exist, and overwrites the content if it does. |
| 'a' | Append. Opens the file for writing, adding new content to the end without overwriting existing data. Creates a new file if it doesn't exist. |

**Read and Write File**

a)To read a file, open it in read mode ("r"), which is the default. You can use various methods to read its content.

**Example**

```
with open("example.txt", "r") as file:
    content = file.read()     # Reads the entire file
```

```
    print(content)
```

b)To write to a file, open it in write mode ("w") or append mode ("a").

**Example**

```
with open("newfile.txt", "w") as file:
    file.write("Hello, Python!\n")

# Append mode ("a") - adds to the end
with open("newfile.txt", "a") as file:
    file.write("This is a new line.\n")
```

## 2.Exception

- An exception is an event that occurs during the execution of a program and disrupts its normal flow.
- Exceptions are runtime errors that can be handled using try and except blocks to prevent the program from crash

**Key components**

I. **try**: This block contains the code that might raise an exception.
II. **except**: This block catches and handles the specific exception(s) raised in the try block.
III. **else**: This optional block executes only if no exception occurs in the try block.
IV. **finally**: This optional block always executes, regardless of whether an exception occurred or was handled.

**Program**: Exception with arguments

```
try:
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))
    result = a / b
    print("Result:", result)
except Exception as e:
    print("Error occurred:", e)
finally:
    print("Program Ended")
```

Output: Error case
Enter first number: 10
Enter second number: 0
Error occurred: division by zero
Program Ended

**Program**: User defined Exception

```
# 🔹 Create user-defined exception
class AgeError(Exception):
    pass
try:
    age = int(input("Enter your age: "))
    if age < 18:
        raise AgeError("Age must be 18 or above")
    print("You are eligible")
except AgeError as e:
    print("Error:", e)
```

**Output**:
Enter your age: 15
Error: Age must be 18 or above

**Explanation**:
Use raise to generate error
Use except to handle error
as e → stores error message

**PROGRAM**: Word Count

```
text = input("Enter a sentence: ")
words = text.split()   # split into words
count = len(words)     # count words
print("Number of words:", count)
```

**Output**:
Enter a sentence: Python is very easy
Number of words: 4

**PROGRAM**: File Copy

```
f1 = open("source.txt", "r")
f2 = open("copy.txt", "w")
data = f1.read()   # read content
f2.write(data)     # write content
print("File copied successfully")
f1.close()
f2.close()
```

FINAL ANSWERS
🔹 1. Target Audience & Use Case

👉 Primary Users:

General public
Women safety
Students
Travelers

👉 Emergency Types:

Physical threat ⚠️
Medical emergency 🏥
Accidents 🚗
Disaster situations 🌪️

💡 Based on your backend + screens, your app is clearly:
👉 “All-in-one emergency response system”

🔹 2. Top 2 Most Important Features (🔥 VERY IMPORTANT)

👉 Highlight ONLY these in demo:

🥇 Feature 1:

👉 SOS Trigger + Real-Time Alert System

One tap SOS
Sends location
Notifies nearby volunteers
🥈 Feature 2:

👉 Live Tracking + Response System

Victim location tracking
Volunteer accepts request
Help is on the way

💡 This matches your backend flow (trigger → alert → response)

🔹 3. Design Style Preference

👉 Choose:
✅ Cross-platform hybrid (BEST)

👉 And:
✅ Keep your dark modern UI

Why?

Your current UI is already premium + hackathon attractive
Dark theme = emergency apps feel serious + urgent
🔹 4. Internet Connectivity

👉 Answer:

YES, internet required for:
Sending SOS
Fetching volunteers
Live tracking
ALSO show:
✅ Offline fallback (Phase Z concept)

💡 Your Phase Z backend clearly includes multi-channel communication

👉 So say:

“Works even in low network using fallback channels”
🔹 5. User Roles

👉 YES — your app has:

Victim (main user)
Volunteer
Verified Volunteer
Admin (optional)

👉 Demo:
✅ Show Victim + Volunteer only

👉 Don’t show admin (waste of time in hackathon)

🔹 6. Backend Data & API Integration

👉 Answer:

Simulate API calls with:
Loading screen
Success response

👉 Data to show:

Emergency ID
Location
Nearby responders
Response status

💡 Your backend clearly follows event/API structure

🔹 7. Error Handling (VERY IMPRESSIVE 🔥)

👉 Show these:

No volunteers → “Searching more responders…”
GPS fail → “Retrying location…”
Network lost → “Switching to fallback mode…”
False SOS → “Cancel alert option”

💡 Your architecture includes error flow system

🔹 8. Demo Flow Priority (🔥 WINNING FLOW)

👉 Choose:

✅ Option C: Show BOTH roles

🎬 PERFECT DEMO FLOW (USE THIS)
Step 1:

Victim opens app → presses SOS

Step 2:
Alert sent
Loading animation
Step 3:

Volunteer receives alert

Step 4:

Volunteer accepts

Step 5:

Victim sees:
👉 “Help is coming” + live tracking

🧠 FINAL STRATEGY (VERY IMPORTANT)

👉 Your app strength is:

Real-world impact 💥
Strong backend architecture 💻
Premium UI 🎨
💥 ONE LINE YOU CAN SAY TO JUDGES

👉
“HumanNet is a real-time decentralized emergency response system that connects victims with nearby responders instantly, even in low network conditions.”

🔥 FINAL TIP

FULL WORKING PROTOTYPE FLOW (Figma clickable demo) that looks like a real app 🚀

This is exactly what you show judges

🧭 🎯 COMPLETE PROTOTYPE FLOW (CLICKABLE)

We’ll design 3 main parts:

📲 Onboarding (Download + Register)
🚨 Emergency Trigger (SOS system)
🤝 Response System (Nearby help via app/Bluetooth/WiFi)
🟢 PART 1: APP DOWNLOAD & REGISTRATION FLOW
Screens:
1. Splash Screen

👉 Button: “Start”

➡️ Click → Setup Screen

2. Setup / Registration Screen

Fields:

Name
Phone
Emergency contacts

👉 Toggle:

Enable Location
Enable Bluetooth/WiFi

👉 Button: “Continue”

➡️ Click → Permission Screen

3. Permission Screen

Show:

Location access
Bluetooth access
Internet access

👉 Button: “Allow & Continue”

➡️ Click → Home (SOS Screen)

🔴 PART 2: SOS TRIGGER FLOW (MAIN FEATURE)
4. Home Screen (SOS Screen)

👉 BIG RED BUTTON: “SOS”

Actions when clicked:

➡️ Navigate → Emergency Loading Screen

5. Emergency Loading Screen

Show:

“Sending Alert…”
Animation (pulse / loading)

Simulate:

GPS fetching
Network sending

➡️ After 2–3 sec → Emergency Sent Screen

6. Emergency Sent Screen

Show:

✅ Alert sent
📍 Location shared
🔍 Searching nearby responders

👉 Button: “View Live Status”

➡️ Click → Tracking Screen

📡 PART 3: NEARBY RESPONSE SYSTEM (IMPORTANT 🔥)
7. Volunteer Alert Screen (Second flow)

👉 Simulate another user (very important)

Show:

Incoming alert
Distance (e.g., 0.8 km away)
Emergency type

👉 Buttons:

Accept
Ignore

➡️ Click “Accept” → Navigation Screen

8. Tracking Screen (Victim Side)

Show:

Map (mock UI)
“Volunteer is coming”
Distance decreasing

👉 Add:

Chat button
Call button
📶 NETWORK SIMULATION (VERY IMPORTANT FOR JUDGES)

👉 Add this logic visually:

Show options in UI:
Internet mode 🌐
Bluetooth mode 📡
WiFi Direct 📶

👉 In prototype:

Create a small popup:

“Network unavailable → Switching to offline mode”

➡️ Then show:

“Connecting via nearby devices…”

💡 This represents your backend Phase Z concept

⚠️ MULTIPLE EMERGENCY TYPES
Add selection screen BEFORE SOS:
Emergency Type Screen

Options:

🚨 Danger
🏥 Medical
🚗 Accident

➡️ Select → Go to SOS flow

🎬 FINAL DEMO FLOW (WHAT YOU PRESENT)
🔥 SCENE 1 (User)
Open app
Register
Click SOS
🔥 SCENE 2 (System)

Alert sent
Searching responders
🔥 SCENE 3 (Volunteer)
Receives alert
Accepts
🔥 SCENE 4 (Result)
Live tracking
“Help is on the way”
🎨 FIGMA CLICK MAPPING (IMPORTANT)
Button Connections:
Start → Setup
Continue → Permissions
Allow → Home
SOS → Loading
Loading → Sent
View Status → Tracking
Accept (volunteer) → Navigation
💡 EXTRA WOW FEATURES (ADD IF TIME)

👉 Add:

Voice trigger (“Help!”) screen
Countdown before SOS
Cancel SOS option
🧠 BACKEND SIMULATION LOGIC (SIMPLE)

Show like:

User clicks SOS
App gets location
Sends alert
Nearby devices receive (Bluetooth/WiFi)
Volunteer accepts
Tracking starts